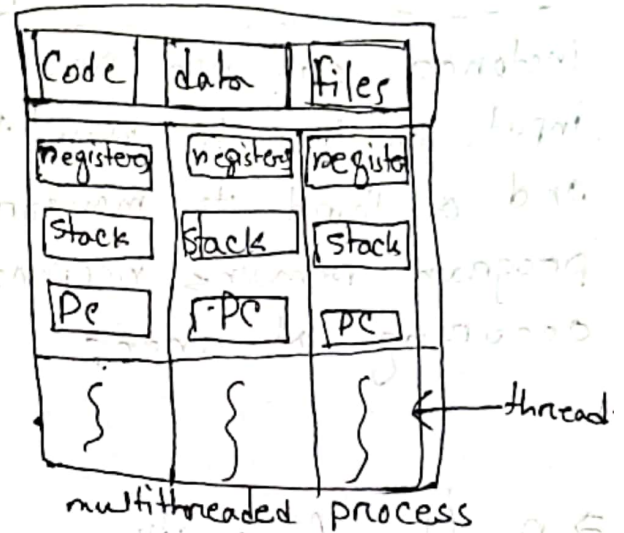
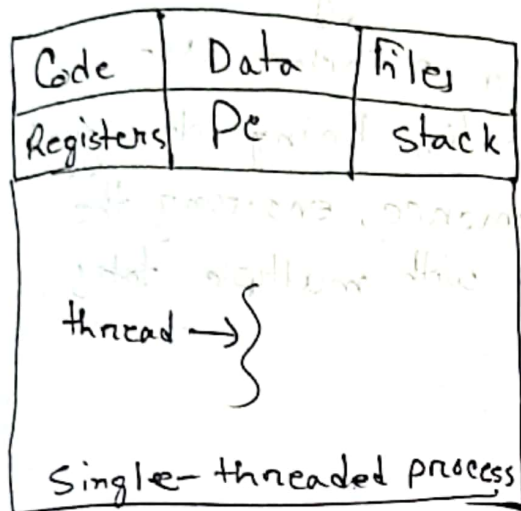


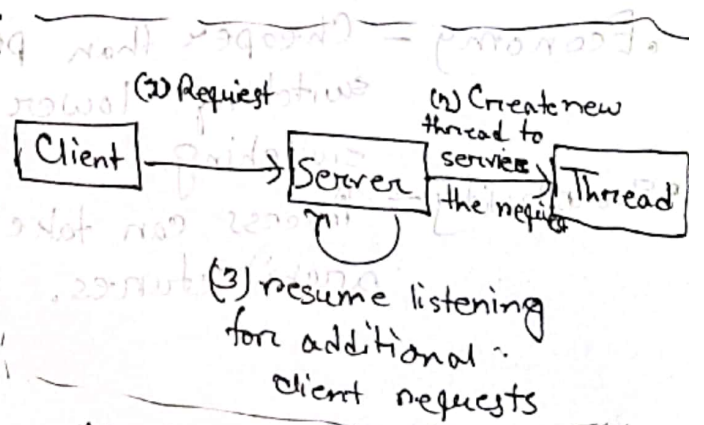
Chapter: 9 - Threads & Concurrency

A thread in an OS is the smallest, lightest unit of the execution within a process.



Three programming examples in which multithreading provide better performance than a single-threaded solution.

① Web Server: A web server can service each client request in a separate thread, allowing it to handle multiple request simultaneously and improving overall performance compared to a single-threaded approach.



Multithreaded server architecture

② Parallelized Matrix Multiplication: Multithreading allows different parts of a matrix to be processed in parallel, significantly speeding up operations like

i) matrix multiplication when compared to a single-threaded solution.

③ Interactive GUI Program: A debugger, for instance, can use one thread to monitor user input, another to run the application being debugged, and a third to monitor performance, ensuring the program remains responsive even with multiple tasks occurring at once.

⇒ Benefits of Multithreads Server:

- Responsiveness - May allow continued execution if part of process is blocked.
- Resource sharing - threads share resources of process easier than shared memory.
- Economy - Cheaper than process creation, thread switching lower overhead than context switching.
- Scalability - Process can take advantage of multicore architectures.

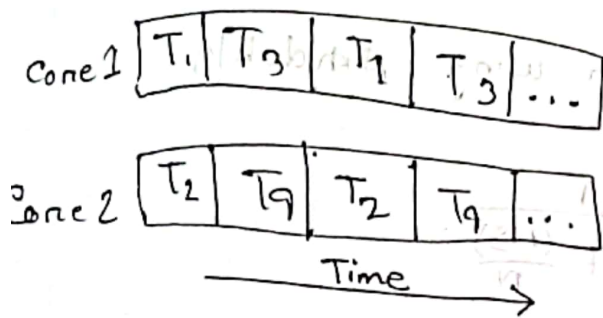
Multicore or multiprocessor system putting pressure on programmers, challenges includes

1. Dividing activities
2. Balance
3. Data splitting
4. Data dependency
5. Testing & Debugging.

⑩ Parallelism VS Concurrency

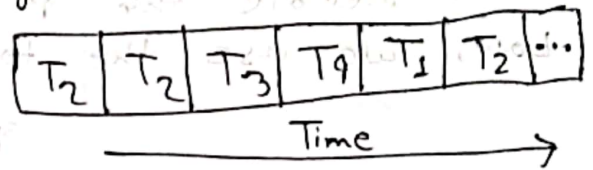
Parallelism: - implies a system can perform more than one task simultaneously.

Concurrency: - supports more than one task making progress. Single processor/core, scheduler providing concurrency.



Parallelism on a multi-core system

single core

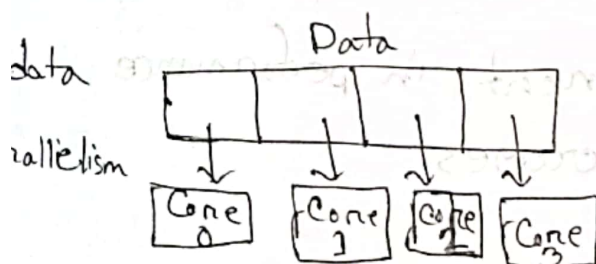


Concurrent execution on single-core system.

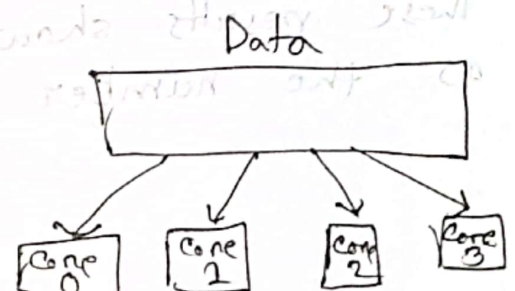
→ Two types of parallelism:

① Data parallelism - distributes subsets of the same data across multiple cores, same operation on each.

② Task parallelism - distributing threads across cores, each thread performing unique operation.



Task Parallelism



Amdahl's Law:

Identify performance gains from adding additional cores to an application that has both serial and parallel components.

→ S is serial portion

→ N processing cores

$$\text{speedup} \leq \frac{1}{S + \frac{(1-S)}{N}}$$

Q2 To calculate the speedup gain using Amdahl's Law, we use the formula

$$\text{speedup} = \frac{1}{S + \frac{(1-S)}{N}}$$

where,

- S is the serial portion of the application.
- N is the number of processing cores.

The problem specifies a 60% parallel component, so the serial component $S = 1 - 0.6 = 0.4 / 40\%$

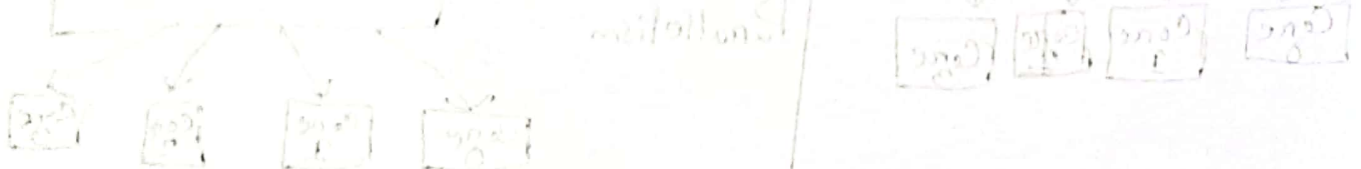
For $N=2$

$$\text{speedup} = \frac{1}{0.4 + \frac{1-0.4}{2}} = 1.43$$

For $N=9$

$$\text{speedup} = \frac{1}{0.4 + \frac{1-0.4}{9}} = 1.82$$

These results show the improvement in performance as the number of core increases.

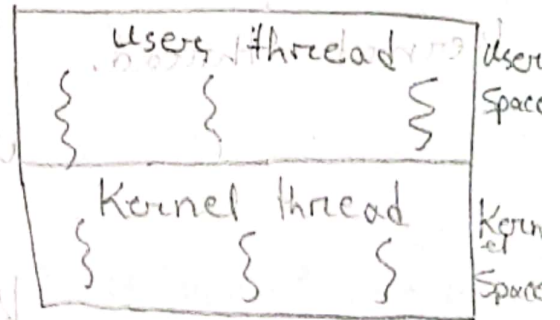


⑩ What are the differences between user-level threads and kernel-level threads? Under what circumstances is one type better than other type?

Aspect	User-level Threads	Kernel-level Thread
Management	Managed by user-level libraries, without OS involvement.	Managed and scheduled directly by the OS
Performance	Faster to create and switch between threads. But cannot take adv advantage of multiple CPUs in parallel. • Pthreads • Windows threads • Java threads	Slower to create and manage, but can run on multiple processors in parallel. • Windows • Linux • Mac OS X • iOS • Android

• User-level threads are better when:

1. Fast thread creation and switching are important.
2. The application runs on a single-core processor without needing parallelism.



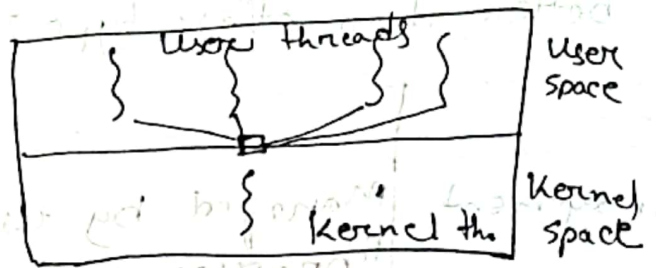
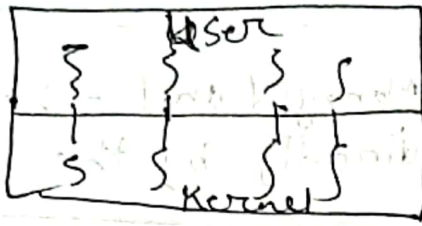
3. There is minimal interaction with the operating system or kernel services.

• Kernel-level threads are better when:

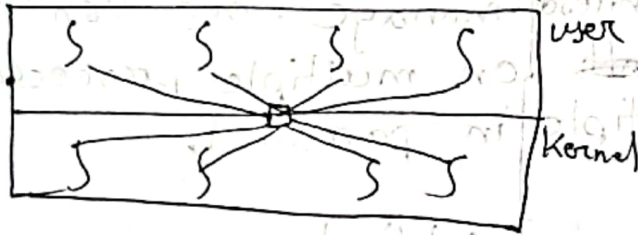
1. The application required true parallelism on multiple cores.
2. Frequent interaction with the operating system, like I/O operations, is necessary.
3. Robust concurrency control is required, managed directly by OS.

2.1 Multithreading Models:

1. ~~One~~ -to-One :

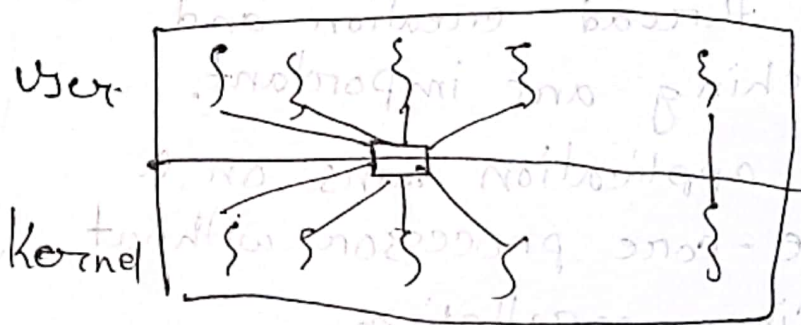


2. Many-to-One



Many-to-Many

① Two-level model : Similar to Many-to-Many except that it allows a user thread to be bound to kernel thread.



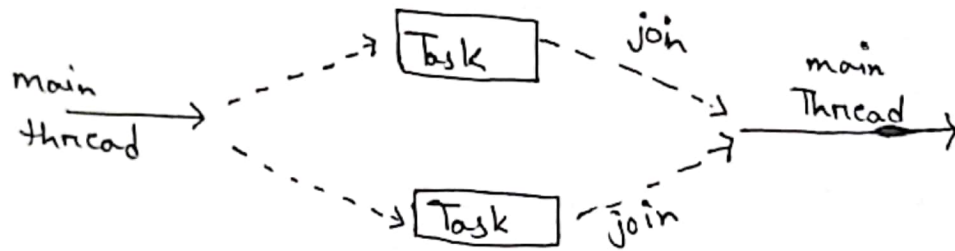
Pthreads : Pthreads refers to the POSIX standard defining an API for thread creation and synchronization.

④ Implicit Threading:

Implicit threading is a concurrency model where the creation, scheduling and management of threads are handled automatically.

Fork-Join Parallelism:

Multiple threads are forked and then joined.



Windows Threads:

A windows application runs as a separate process, and each process may contain one or more threads.

The general component of the thread include:

1. A thread ID uniquely identifying the thread.
2. A register set representing the status of processor.
3. A program counter.
4. Separate user and kernel stacks for when thread runs in user mode or kernel mode.
5. Private data storage area used by run-time libraries and dynamic link libraries.

⇒ The register set, stacks and private storage area are known as the context of the thread.